

AVON · SECURITY REVIEW

Avon Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Avon	Review	Guardian	November 24 – April 30, 2026	May 7, 2026

§ 00

Contents.

64 pages — overview, confidence ranking, scope, 45 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 22 findings	P. 11
	Remediation Review 1 · 8 findings	P. 34
	Remediation Review 2 · 4 findings	P. 43
	Remediation Review 3 · 2 findings	P. 47
	Remediation Review 4 · 8 findings	P. 50
	Remediation Review 5 · 1 finding	P. 61
§ 05	Disclaimer	P. 63
§ 06	About Guardian	P. 64

§ 01

Overview.

Guardian reviewed Review across 6 review rounds. The final report includes 1 High, 18 Medium, 19 Low, 7 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	avon-xyz/vault-contract, avon-xyz/avon-core, avon-xyz/avon-periphery
CLIENT	Avon	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3 · Remediation Review 4 · Remediation Review 5
PROJECT	Review	MAIN REVIEW COMMIT	3495c7a80390
PREPARED BY	Guardian	REMEDATION REVIEW 1 COMMIT	48b3820440ce
AUDIT WINDOW	November 24 – April 30, 2026		
FINAL REPORT	May 7, 2026		

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	1	0	0	0	1
Medium	18	0	3	0	15
Low	19	0	7	1	11
Informational	7	0	3	0	4
All severities	45	0	13	1	31

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `avon-xyz/vault-contract`, `avon-xyz/avon-core`, `avon-xyz/avon-periphery`. The scope includes 3,952 source lines across 32 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/VaultV2.sol	777	1,306
src/adapters/AvonPoolAdapter.sol	114	246
src/adapters/AvonPoolAdapterFactory.sol	16	41
src/Orderbook.sol	363	596
src/OrderbookFactory.sol	82	139
src/OrderbookFactoryStorage.sol	23	28
src/libraries/AugmentedRedBlackTreeLib.sol	635	978
src/libraries/ErrorsLib.sol	28	91
src/libraries/EventsLib.sol	22	113
src/libraries/MathLib.sol	25	44
src/libraries/OrderbookLib.sol	265	429
src/pool/AvonPool.sol	655	903
src/pool/PoolStorage.sol	93	129
src/libraries/LiquidityAllocator.sol	27	44
src/libraries/SharesLib.sol	19	44
src/factory/AvonPoolFactory.sol	51	98
src/factory/VaultFactory.sol	41	81
src/oracle/Oracle.sol	47	73
src/irm/LinearKinkIRM.sol	32	53
src/pool/utils/PoolConstants.sol	28	46
src/pool/utils/PoolErrors.sol	21	62
src/pool/utils/PoolEvents.sol	83	292
src/pool/utils/PoolGetter.sol	94	125
src/pool/extensions/AccrueInterest.sol	56	86
src/pool/extensions/BorrowRepay.sol	89	152
src/pool/extensions/CollateralManagement.sol	26	47
src/pool/extensions/DepositWithdraw.sol	37	77
src/pool/extensions/FlashLoan.sol	33	54
src/pool/extensions/Liquidation.sol	99	154
src/pool/extensions/PositionGuard.sol	30	53
src/pool/extensions/UpdateOrders.sol	35	50
src/pool/extensions/Utils.sol	6	10

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Referrer Claim DoS From Rounding Mismatch	Rounding	MEDIUM	RESOLVED	p. 11
M-02	Inverted Slippage Guard In _borrow Function	Logical Error	MEDIUM	RESOLVED	p. 13
M-03	EIP-4626 Previews Skew With Interest Accrual	Unexpected Behavior	MEDIUM	RESOLVED	p. 14
M-04	Underwater Liquidations Inflate Bad Debt	Logical Error	MEDIUM	RESOLVED	p. 15
M-05	Repay Cleanup Breaks Borrow Share Accounting	DoS	MEDIUM	RESOLVED	p. 16
M-06	Referrer Can Be Forced Via Zero-Share Transfer	Logical Error	MEDIUM	RESOLVED	p. 17
M-07	Over-Seizure When repaidShares Is Clamped	Logical Error	MEDIUM	RESOLVED	p. 18
L-01	Unaccounted Fees When feeRecipient Is Zero	Logical Error	LOW	RESOLVED	p. 19
L-02	Inconsistent Interest Projections While Paused	Logical Error	LOW	RESOLVED	p. 20
L-03	Deposit Cap Not Reflected	Informational	LOW	PARTIALLY RESOLVED	p. 21
L-04	No Slippage Bounds On ERC4626 Flows	Unexpected Behavior	LOW	RESOLVED	p. 22
L-05	Pool Lacks Support	Configuration	LOW	ACKNOWLEDGED	p. 23
L-06	previewBorrow Ignores Borrow Cap	Validation	LOW	RESOLVED	p. 24

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-07	PreviewBorrow Includes Paused Pools	Validation	LOW	RESOLVED	p. 25
L-08	Stale Orders After Borrow Cap Update	Logical Error	LOW	RESOLVED	p. 26
L-09	maxWithdraw and maxRedeem Ignore Pool Constraints	Unexpected Behavior	LOW	RESOLVED	p. 27
L-10	Paused Pools Block Fills During Matching	Logical Error	LOW	RESOLVED	p. 28
L-11	Incorrect loanTokenAmount	Logical Error	LOW	RESOLVED	p. 29
L-12	Market Order LTV And Rate Discrepancy	Unexpected Behavior	LOW	RESOLVED	p. 30
I-01	Redundant Fee Adjustment In Interest Preview	Superfluous Code	INFORMATIONAL	RESOLVED	p. 31
I-02	Incorrect Maker Field In OrderCanceled Event	Events	INFORMATIONAL	RESOLVED	p. 32
I-03	IPoolImplementation Interface Mismatch	Informational	INFORMATIONAL	RESOLVED	p. 33
REMEDATION REVIEW 1					
M-01	maxMint Exceeds Cap After Interest Accrual	Unexpected Behavior	MEDIUM	RESOLVED	p. 34
M-02	Transfers Can Force Referrer Assignment	Logical Error	MEDIUM	RESOLVED	p. 35
M-03	De-whitelisted Referrers Can Still Claim Fees	Logical Error	MEDIUM	RESOLVED	p. 36
L-01	_updateOrders Can Execute During Pause	Unexpected Behavior	LOW	RESOLVED	p. 37
L-02	Share Inflation Can Cause Temporary DOS	DoS	LOW	ACKNOWLEDGED	p. 38
I-01	Misnamed Slippage Parameter In _borrow	—	INFORMATIONAL	RESOLVED	p. 39

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-02	Paused Pool Quotes Can Desync State	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 40
I-03	Shares Sent Without Receive Gate Checks	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 42

REMEDIATION REVIEW 2

M-01	Re-whitelist Restores Stake With Stale Index	Logical Error	MEDIUM	RESOLVED	p. 43
M-02	Withdraw DoS After Referrer De-Whitelist	Logical Error	MEDIUM	RESOLVED	p. 44
M-03	totalReferrerShares Drift On Transfers	Logical Error	MEDIUM	RESOLVED	p. 45
M-04	clearReferrerData Skews totalReferrerShares	DoS	MEDIUM	RESOLVED	p. 46

REMEDIATION REVIEW 3

M-01	Legacy userAttributedShares Wipes New Stake	Logical Error	MEDIUM	ACKNOWLEDGED	p. 47
I-01	Liquidation Can Panic With Zero bonusFactor	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 49

REMEDIATION REVIEW 4

H-01	Residual Limit Orders Brick on Partial Fill	Logical Error	HIGH	RESOLVED	p. 50
M-01	Partial Fills Leave Residual Orders Unbacked	Logical Error	MEDIUM	RESOLVED	p. 52
M-02	Partial Fills Enforce Full-Order Min Output	Logical Error	MEDIUM	ACKNOWLEDGED	p. 54
L-01	No Admin Cancel for Stuck Borrow Orders	Warning	LOW	ACKNOWLEDGED	p. 56
L-02	RedStone maxAge allows stale data	Configuration	LOW	ACKNOWLEDGED	p. 57

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03	Loan/USD freshness is not validated	Configuration	LOW	ACKNOWLEDGED	p. 58
L-04	Underwater Liquidations Lack Incentive	Warning	LOW	ACKNOWLEDGED	p. 59
L-05	Zero repay allows users to discard dust interest	Logical Error	LOW	ACKNOWLEDGED	p. 60

REMEDIATION REVIEW 5

M-01	Stale quote can block borrow matching	Logical Error	MEDIUM	ACKNOWLEDGED	p. 61
------	---------------------------------------	---------------	--------	--------------	-------

MEDIUM **M-01**

RESOLVED

Referrer Claim DoS From Rounding Mismatch

CATEGORY Rounding
LOCATION VaultV2.sol
REVIEW Main Review

DESCRIPTION

The referrer rewards system relies on an index-based accounting model to distribute escrowed referrer fee shares. The mechanism allocates newly escrowed shares by increasing a global index (`referrerIndex`) and tracking how many shares have been globally allocated but not yet claimed in `totalUnclaimedReferrerShares` . Both values are updated during index distribution which performs two separate floor-rounded conversions

```
uint256 indexIncrease = pending.mulDivDown(WAD, stake);  
referrerIndex += indexIncrease;  
totalUnclaimedReferrerShares += indexIncrease.mulDivDown(stake, WAD);
```

A referrer's personal accrual is computed differently. It accrues new rewards based on the full index delta since their last interaction and apply floor rounding only once.

```
uint256 delta = idx - lastIdx;  
uint256 sharesAccrued = stake.mulDivDown(delta, WAD);  
if (sharesAccrued != 0) {  
    referrerAccrued[referrer] += sharesAccrued;  
}
```

The issue arises because `totalUnclaimedReferrerShares` is updated by rounding down twice on every index update. A referrer, however, accrues rewards by rounding down only once on the total index change since their last interaction. Because there's a mismatch between the sum of round-downs and rounding down the sum, the referrer's computed claimable amount can end up slightly higher than `totalUnclaimedReferrerShares` , even though the rewards were escrowed.

When a referrer calls `claimReferrerFees` , the contract subtracts the referrer's claimable shares from `totalUnclaimedReferrerShares` . If the referrer's claimable amount is greater than `totalUnclaimedReferrerShares` , the subtraction underflows and the transaction reverts. As a result, affected referrers will be unable to claim their rewards.

M-01 Referrer Claim DoS From Rounding Mismatch

CONTINUED

— RECOMMENDATION

Carry forward the discarded fractional remainder in each index update and transfers only the safely claimable amount instead of reverting when the computed claim exceeds `totalUnclaimedReferrerShares`. Optionally, consider allowing referrer to provide the intended amount of referrer fee.

PROOF OF CONCEPT https://github.com/GuardianOrg/vault-contract-team1-1763752225872/tree/referrer_rounding_poc

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/vault-contract/commit/48b3820440ce8408ba67ab4f61eb24f6ecb13227>

MEDIUM **M-02**

RESOLVED

Inverted Slippage Guard In `_borrow` Function

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/extensions/BorrowRepay.sol#L51
REVIEW	Main Review

DESCRIPTION

In the borrow path, when the exact amount of asset is provided, the internal `_borrow` path computes the borrower's debt shares from the requested assets and then enforces that the amount of shares must be greater or equal to `minExpected`, treating this as a minimum tolerable number of debt shares. Since debt shares represent liability, this check is inverted - it causes transactions to revert when the user would receive fewer shares than expected (better terms) while allowing execution when they receive more shares than expected (worse terms), meaning `minExpected` cannot be used to cap the borrower's maximum debt and instead only blocks favorable outcomes.

RECOMMENDATION

Replace the minimum shares check with a maximum debt-share cap so the call reverts whenever calculated debt shares exceed that bound.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/34327462455fca208ef8215042cb65171dd17370>

MEDIUM **M-03**

RESOLVED

EIP-4626 Previews Skew With Interest Accrual

CATEGORY Unexpected Behavior
LOCATION AvonPool.sol
REVIEW Main Review

DESCRIPTION

In AvonPool, the state-changing `deposit` and `mint` functions call `accrueInterest` before delegating to the counterpart functions defined in ERC4626, while `previewDeposit` and `previewMint` are only inherited and do not account for this interest accrual. This means that within a single transaction an integrator can observe `previewDeposit` output on pre-accrual state and then call `deposit`, which will mint fewer shares than the preview predicted. Similarly, `previewMint` can return fewer required assets than `mint` actually pulls after accrual.

This violates EIP-4626's MUST-level requirements that, in the same transaction, `deposit` must always return the same or more shares than `previewDeposit` and `mint` must always consume the same or fewer assets than `previewMint`, and can mislead or break integrators relying on these bounds for slippage and safety checks.

RECOMMENDATION

Override `previewDeposit` and `previewMint` to use the same preview-accrued totals that `deposit` and `mint` use after `accrueInterest`, keeping the previews consistent with actual execution.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/d04d0b3d50fee70bbc69b48df00d737df5f87dd6>

MEDIUM **M-04**

RESOLVED

Underwater Liquidations Inflate Bad Debt

CATEGORY Logical Error
LOCATION Liquidation.sol
REVIEW Main Review

DESCRIPTION

The liquidation mechanism allows partial liquidations of underwater positions to decrease their health score, meaning each liquidation step removes more collateral value than debt value repaid when the liquidation bonus exceeds the current collateral-to-debt ratio.

Health score is defined as $(LLTV * collateralValue) / borrowAssets$ and a liquidation that repays some amount of debt seizes collateral worth $bonusFactor * debt$. When the liquidation bonus factor is greater than $collateralValue / borrowAssets$, the collateral value drops faster than the debt, so the post-liquidation health score will be lower than before.

As a result, the collateral/debt coverage ratio gets strictly worse after a partial liquidation, so if prices continue to move against the collateral, this will increase the amount of bad debt ultimately socialized to LPs. This behaviour contradicts the invariant that liquidation should improve a position's health, and creates opportunity to game this mechanism by executing small partial liquidations that push positions deeper underwater, amplifying future bonuses and extracting more value from borrower collateral and worsening the eventual losses absorbed by LPs.

RECOMMENDATION

Clamp the effective liquidation bonus to at most the current collateral-to-debt ratio, so a partial liquidation can only keep the health score the same or improve it.

PROOF OF CONCEPT https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/tree/partial_liquidation_worsens_healthScore_poc

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/b348bd5887881ea10fad3e1301f7c28705ea1a23>

MEDIUM **M-05**

RESOLVED

Repay Cleanup Breaks Borrow Share Accounting

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/extensions/BorrowRepay.sol#L130
REVIEW	Main Review

— DESCRIPTION

The repay function includes a cleanup condition that sets `s.totalBorrowShares` to zero whenever `s.totalBorrowShares` is greater than zero while `s.totalBorrowAssets` is zero. This is intended to prevent leaving residual global borrow shares after all borrow assets have been fully repaid.

However, due to the rounding direction in repay and liquidation flows, users can slightly overpay assets, causing the system to reach a state where the cleanup branch zeroes out `s.totalBorrowShares` even though at least one position still holds non-zero `borrowShares`.

This creates an accounting desync: the sum of all user `borrowShares` becomes strictly greater than the global `totalBorrowShares`. Once the system enters this state, any attempt to fully repay or liquidate the final remaining borrower will revert due to an underflow when subtracting from `totalBorrowShares`.

— RECOMMENDATION

Update or remove the cleanup logic so that `s.totalBorrowShares` is only zeroed when both global borrow assets and all per-position `borrowShares` have reached zero.

PROOF OF CONCEPT <https://github.com/GuardianOrg/vault-contract-fuzz-1763752232141/blob/9465b398b5b416f83d38bac922abe492f070c6fd/test/fuzzing/FoundryPlayground.sol#L1836>

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/0ec384ba05aa376b91443f71f0d486d3e6e4243c>

MEDIUM **M-06**

RESOLVED

Referrer Can Be Forced Via Zero-Share Transfer

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/c934359ad432cfb1355661c8f8c666970b0187b1/src/VaultV2.sol#L1133
REVIEW	Main Review

— DESCRIPTION

The transfer function updates referrer associations whenever shares move between accounts. If the recipient has no referrer set and the sender has a whitelisted referrer, the recipient automatically inherits the sender's referrer.

This behaviour can be abused to force a referrer onto any user. An attacker can simply send a zero-share transfer to a user who has not yet set a referrer. That user is then assigned the attacker's referrer. When the user later performs a deposit and specifies their own referrer through the `depositWithReferrer` function, the protocol ignores the supplied referrer because the user already has one set. Since referrers cannot be changed once assigned, the user is permanently locked with the initial referrer.

— RECOMMENDATION

Restrict referrer assignment so it only occurs on explicit referrer-setting calls. This prevents attackers from forcing referrers via zero-value or dust transfers.

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/vault-contract/commit/a36cc81c66eb947c2c856b67eba40622dd5c9144>

MEDIUM **M-07**

RESOLVED

Over-Seizure When repaidShares Is Clamped

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/extensions/Liquidation.sol#L23
REVIEW	Main Review

— DESCRIPTION

In the `_liquidate` function, when liquidating with the `seizedAssets` parameter, the function first derives `repaidShares` from the proposed `seizedAssets` (after applying `seizeCap`) using the configured liquidation bonus. Later, it clamps `repaidShares` to `position.borrowShares`, but does not recompute `seizedAssets` after this clamp.

As a result, in edge cases where the initial calculation would repay more than the remaining debt, the liquidator still seizes the full `seizedAssets` amount while only repaying the reduced, clamped debt. This allows a liquidator to over-seize collateral beyond what corresponds to the final repaid debt and bonus, causing additional value loss for the borrower.

— RECOMMENDATION

Whenever `repaidShares` is reduced (for example, clamped to `position.borrowShares`) in the `seizedAssets` path, recompute `seizedAssets` from the final `repaidShares`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/d87248d2eb9304cf05005977accd5ca0f4dffb/test/unit/LiquidationTest.t.sol#L139>

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/1dbd249d5594e7d01f74e095c208ebff16bde00a>

LOW **L-01**

RESOLVED

Unaccounted Fees When feeRecipient Is Zero

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/extensions/FlashLoan.sol#L66
REVIEW	Main Review

DESCRIPTION

In `_flashLoanLoanToken`, the total flashloan fee is split into `poolFeeAmount` and `protocolFeeAmount` and only `poolFeeAmount` is added to `s.totalSupplyAssets`. If `protocolFeeAmount` is greater than 0 but `feeRecipient` is `address(0)`, the transfer is skipped and the full `feeAmount` stays in the vault's token balance, but only `poolFeeAmount` is reflected in `s.totalSupplyAssets`. This leaves orphaned tokens inside the pool that are neither claimable by LPs nor withdrawn as protocol fees, breaking the expected accounting invariant and effectively locking part of the flashloan fees inside the contract.

RECOMMENDATION

Ensure the protocol fee portion is consistently accounted by either adding `protocolFeeAmount` to `s.totalSupplyAssets` whenever it remains in the pool instead of being transferred out or reverting when `protocolFlashLoanFeePercentage > 0 && feeRecipient == address(0)`.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/pull/42/changes/0b3254493a44ac54e0f85c85aedaf79e9b6d685>

LOW **L-02**

RESOLVED

Inconsistent Interest Projections While Paused

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/Utils/PoolGetter.sol#L92
REVIEW	Main Review

DESCRIPTION

When the pool is paused, interest accrual is applied to the pause timestamp and then blocked, but `_previewAccrueInterest` and the view functions that rely on it continue to compute interest using `elapsed = block.timestamp - s.lastUpdate` as if the pool were still accruing normally. On unpause, `pausePool` function sets `s.lastUpdate = block.timestamp` without applying that projected interest, so all interest previewed during the paused interval is effectively discarded. This creates a discrepancy where views overestimate debt and yield while paused, and then return to lower, actually-accrued values once the pool is resumed.

RECOMMENDATION

Make preview logic respect the paused state by treating `elapsed` as zero while the pool is paused.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/ceebb18663d6ac75e5e5ff2ffcc678615fdb1d72>

LOW **L-03**

PARTIALLY RESOLVED

Deposit Cap Not Reflected

CATEGORY Informational
LOCATION ERC4626.sol
REVIEW Main Review

DESCRIPTION

AvonPool enforces deposit cap inside the deposit and mint functions, via an internal check. However, the contract inherits the default ERC4626 implementations of `maxDeposit` and `maxMint`, which both return `type(uint256).max`. As a result, off-chain integrators that follow the standard pattern of querying `maxDeposit(receiver)` / `maxMint(receiver)` to determine the maximum safe amount will be told they can deposit/mint arbitrarily large amounts, even though such calls can revert at runtime once the internal cap check is hit.

RECOMMENDATION

Override `maxDeposit` and `maxMint` in AvonPool so they reflect the effective `depositCap` or clearly document for integrators that these view functions do not enforce the pool's deposit cap.

RESOLUTION

Status: Partially resolved.

<https://github.com/avon-xyz/avon-periphery/commit/b8feb46b52636af412fbe1fc4f2a638951b30c1c>

LOW **L-04**

RESOLVED

No Slippage Bounds On ERC4626 Flows

CATEGORY Unexpected Behavior
LOCATION AvonPool.sol
REVIEW Main Review

DESCRIPTION

AvonPool exposes the standard ERC4626 `deposit`, `mint`, `withdraw`, and `redeem` functions but does not provide wrapper functions that allow callers to specify minimum/maximum bounds on shares or assets, even though EIP4626 recommends such variants when supporting direct EOA usage.

As a result, users and integrators have no built-in way to protect themselves from slippage and unexpected exchange rate changes on these operations, increasing the risk of unintentionally receiving fewer assets or shares than intended.

RECOMMENDATION

Consider adding slippage-aware wrapper functions for `deposit`, `mint`, `withdraw`, and `redeem` that accept min/max asset or share bounds and revert if these bounds are violated.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/d24797dd8373c5856a45792c0ba285f0dba6f58e>

LOW **L-05**

ACKNOWLEDGED

Pool Lacks Support

CATEGORY Configuration
LOCATION AvonPool.sol
REVIEW Main Review

— DESCRIPTION

The Avon pool contract does not support non-standard ERC20 tokens. For instance tokens that take fees on transfer, since those fees break the expected accounting. In addition, tokens with callbacks can trigger reentrancy during deposit or borrow operations and bypass internal caps, allowing users to manipulate balances or exceed intended limits.

— RECOMMENDATION

Ensure the pool contract only accepts standard ERC20 tokens.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

previewBorrow Ignores Borrow Cap

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/utis/PoolGetter.sol#L54>

REVIEW Main Review

DESCRIPTION

The previewBorrow function does not account for the protocol's borrowCap when estimating how much a user can borrow. As a result, callers relying on previewBorrow may receive a borrow estimate that exceeds the actual allowable amount, causing transactions to revert when the real borrow call enforces the cap.

RECOMMENDATION

Update previewBorrow to include the borrow-cap constraint so previews match the limits enforced during actual borrowing.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/ce827b79b7ffd59870f0c8d5751e5dc546948820>

LOW **L-07**

RESOLVED

PreviewBorrow Includes Paused Pools

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/20c33fcb3be244c57dc027d6a2660a1db205f4c0/src/Orderbook.sol#L412
REVIEW	Main Review

— DESCRIPTION

The previewBorrow function doesn't skip paused pools, even though the matching logic ignores them during actual matching execution. As a result, previewBorrow can return estimates that rely on liquidity from pools that will not be used, causing the previewed results to differ from real outcomes.

— RECOMMENDATION

Update previewBorrow to filter out paused pools in the same way the matching logic does, ensuring previews align with executable behaviour.

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/be7594bbe9aeeb79fc50ffa0d5905a2c41b3b1db>

LOW **L-08**

RESOLVED

Stale Orders After Borrow Cap Update

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/AvonPool.sol#L736
REVIEW	Main Review

— DESCRIPTION

The `_executeUpdateBorrowCap` function updates the borrow cap, which is then used by `_updateOrders` to compute available liquidity for quoting and maintaining the orderbook. Because `_executeUpdateBorrowCap` does not call `_updateOrders` after applying the new cap, the existing orders remain based on outdated liquidity assumptions. This can leave quotes in the orderbook that are no longer fillable under the updated cap.

— RECOMMENDATION

Call `_updateOrders` after applying a borrow cap update so the orderbook reflects the new available liquidity.

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/c9a221ae05f4c52d4480ab4c6379465565a300cf>

LOW **L-09**

RESOLVED

maxWithdraw and maxRedeem Ignore Pool Constraints

CATEGORY	Unexpected Behavior
LOCATION	AvonPool.sol
REVIEW	Main Review

DESCRIPTION

AvonPool adds extra withdrawal constraints (liquidity checks against `totalBorrowAssets` and the `whenNotPaused` modifier) in withdraw and redeem function, but inherits the default ERC-4626 implementations of `maxWithdraw` and `maxRedeem`, which only look at the user's share balance and the conversion rate. As a result, `maxWithdraw` and `maxRedeem` can report amounts that exceed what the pool actually can serve (when liquidity is constrained by borrows or the pool is paused), breaking the EIP-4626 requirement that these functions must factor global limits and never return a value higher than the actual maximum and skewing integrators' assumptions when they treat these as safe upper bounds.

RECOMMENDATION

Override `maxWithdraw` and `maxRedeem` so they incorporate liquidity constraints and pause state.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/39c5230ee1d3b0a98bcd53dc08520a8eabb6c292>

LOW **L-10**

RESOLVED

Paused Pools Block Fills During Matching

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/20c33fcb3be244c57dc027d6a2660a1db205f4c0/src/Orderbook.sol#L260
REVIEW	Main Review

DESCRIPTION

`matchMarketBorrowOrder` and `matchLimitBorrowOrder` both skip paused pools when calling `_aggregatePoolData`, ensuring that paused pools do not contribute liquidity to the final borrow result. However, earlier in the flow, both functions call `_matchOrder`, which iterates through lender orders and attempts to fill the borrower's request.

The issue is that `_matchOrder` does not check whether a pool is paused. If it encounters an order from a paused pool, it will attempt to match it and count that amount toward the borrower's progress. Later, `_aggregatePoolData` correctly excludes that pool, causing the matched amount to be discarded.

As a result, a borrower may receive only a partial fill even when sufficient liquidity exists in active pools. If the borrower uses a loose `minAmountExpected`, the call may still succeed but unnecessarily charge the flat matching fee despite having no actual fill. In cases where a paused pool has deep liquidity and good rates but its orders remain on the book, it can consistently block full matches.

In principle, pools should cancel all orders before pausing. However, if cancellation fails or an order remains for any reason, the stale order can still be picked up by `_matchOrder`, triggering this inconsistency.

RECOMMENDATION

Ensure `_matchOrder` excludes paused pools in the same way `_aggregatePoolData` does, so that paused pools cannot interfere with match progression.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/be7594bbe9aeeb79fc50ffa0d5905a2c41b3b1db>

LOW **L-11**

RESOLVED

Incorrect loanTokenAmount

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/20c33fcb3be244c57dc027d6a2660a1db205f4c0/src/Orderbook.sol#L452
REVIEW	Main Review

DESCRIPTION

In `previewBorrow` function, when `previewBorrowParams.isCollateral == false`, the function correctly computes `PreviewMatchedOrder` and `collateralRequired` from the matched orders, but never updates `loanTokenAmount`, leaving it at its default value of 0 even when `totalMatched` is greater than 0, which makes the returned data inconsistent with the actual matches and can mislead integrators that rely on `loanTokenAmount` to understand how much loan liquidity is actually available. The function's NatSpec explicitly states that `loanTokenAmount` is "The total amount of loan tokens that would be received", which is also meaningful in the `isCollateral == false` case, and returning 0 there is inconsistent with that documented behavior.

RECOMMENDATION

Consider updating `previewBorrow` so that `loanTokenAmount` is always set to the total matched loan amount in both modes or ensure the NatSpec explicitly documents this behavior so integrators can safely rely on the return value.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/9a6baa3b214969522478a1b81b1bc36a727164d0>

LOW **L-12**

RESOLVED

Market Order LTV And Rate Discrepancy

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/main/src/Orderbook.sol#L424 https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/main/src/Orderbook.sol#L257
REVIEW	Main Review

DESCRIPTION

For market orders, `previewBorrow` unconditionally overwrites the caller supplied LTV with `MIN_LTV` (50%), and `rate` with the highest lender rate, while `matchMarketBorrowOrder` only defaults to `MIN_LTV` and the highest lender rate when `ltv` or `rate` are set to 0, otherwise honoring custom non-zero value. As a result, previews can be computed with different `rate / ltv` constraints than the ones actually used at execution, leading to incorrect collateral estimates and different pools/liquidity being matched than previewed, causing confusing UX and potential failed or unexpected transactions for users and integrators that rely on `previewBorrow` as an accurate execution simulator.

RECOMMENDATION

Update `previewBorrow` so that for market orders, it resolves `ltv` and `rate` exactly like `matchMarketBorrowOrder` (only defaulting `ltv / rate` when the input value is 0), or clearly document that `isMarketOrder==true` ignores any caller-supplied rate/ltv.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/c789959bceec1f7d6e13e4e42d41e12e908a3ae8>

INFORMATIONAL I-01

RESOLVED

Redundant Fee Adjustment In Interest Preview

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/1e58b0545051e38a9cb34aded7bab3fb98995f16/src/pool/utils/PoolGetter.sol#L133-L135
REVIEW	Main Review

DESCRIPTION

The protocol uses `_accrueInterest` function to update state and `_previewAccrueInterest` for view-only projections, both following the same pattern - compute gross `accruedInterest`, add it to `totalBorrowAssets` and `totalSupplyAssets`, derive `managerFeeAmount` and `protocolFeeAmount` and mint fee shares. However, `_previewAccrueInterest` contains an extra block.

```
if (managerFeeAmount > 0 || protocolFeeAmount > 0) {  
    accruedInterest -= (managerFeeAmount + protocolFeeAmount);  
}
```

This modifies only a local `accruedInterest` that is never used afterward, so it has no effect on the current previewed totals and is misleading.

RECOMMENDATION

Remove the unused fee subtraction in `_previewAccrueInterest` function.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/d9b95a625314c5d357d37d1808dbe383b195886b>

INFORMATIONAL I-02

RESOLVED

Incorrect Maker Field In OrderCanceled Event

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/main/src/libraries/OrderbookLib.sol#L167
REVIEW	Main Review

DESCRIPTION

In `_cancelOrder`, the `OrderCanceled` event is emitted as `emit EventsLib.OrderCanceled(isLender, msg.sender, rate, ltv, amount)`, even though the actual order owner is stored in `entry.account`. This misattributes cancellations triggered by owners/keepers/pool cleanup to the caller rather than the original maker, reducing the reliability of the event log for auditing.

RECOMMENDATION

Emit `OrderCanceled` with `entry.account` as the `maker` argument.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/b80922ccba9967e12c1c98317ba9d502a5de9dc9>

INFORMATIONAL I-03

RESOLVED

IPoolImplementation Interface Mismatch

CATEGORY	Informational
LOCATION	IPoolImplementation.sol
REVIEW	Main Review

DESCRIPTION

The IPoolImplementation interface used in the Orderbook repo does not match the IPoolImplementation actually implemented by AvonPool, which may lead to broken integration for several core methods in the future.

- `liquidate` is declared in the Orderbook interface as `liquidate(address borrower, uint256 assets, uint256 shares)` but AvonPool implements `liquidate(address borrower, uint256 assets, uint256 shares, uint256 minSeizedAmount, uint256 maxRepaidAsset, bytes calldata data)`,
- `updateOrderbook` is declared as `updateOrderbook(address newOrderbook, address newOrderbookFactory)` in the Orderbook interface, while AvonPool expects an additional `bytes32 salt` argument,
- The Orderbook interface exposes `increaseLLTV(uint64 newLTV)` whereas AvonPool uses `updateLLTVUpward(uint64 newLTV, bytes32 salt)`

RECOMMENDATION

Update the Orderbook's IPoolImplementation to match AvonPool's signatures.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-core/commit/6ed55231f6d9e84f501ca0f073fc2b46c76212ce>

MEDIUM **M-01**

RESOLVED

maxMint Exceeds Cap After Interest Accrual

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/main/src/pool/AvonPool.sol#L229>

REVIEW Remediation Review 1

DESCRIPTION

The `maxDeposit` uses `_previewAccrueInterest(false)` to compute remaining deposit capacity from post-accrual `totalSupplyAssets`, but `maxMint` converts that capacity into shares with `convertToShares`, which uses pre-accrual pool totals. This mismatch can cause `maxMint` function to return a share amount that is not actually mintable once mint flow calls `accrueInterest` and recomputes required assets, leading to unexpected reverts and breaking ERC-4626 integrator assumptions that `maxMint` is a reliable upper bound.

RECOMMENDATION

Update `maxMint` to derive shares using the same preview-accrued totals as `maxDeposit`.

PROOF OF CONCEPT https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/tree/max_mint_inconsistency_poc

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/3b4636a62ae9d517e2d63b418d679272d8474d16>

MEDIUM **M-02**

RESOLVED

Transfers Can Force Referrer Assignment

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/48b3820440ce8408ba67ab4f61eb24f6ecb13227/src/VaultV2.sol#L1135
REVIEW	Remediation Review 1

DESCRIPTION

The `_updateReferrerOnTransfer` function was updated to avoid assigning a referrer on zero-amount transfers. However, the issue still exists. If a recipient does not yet have a referrer set, and the sender has a whitelisted referrer, transferring any non-zero amount will cause the recipient to inherit the sender's referrer.

This makes it possible for an attacker to force a referrer onto any address at minimal cost by sending a dust transfer. Once this happens, even if the recipient later attempts to set their own referrer explicitly through functions like `depositWithReferrer`, the forced referrer will take precedence and cannot be overridden.

RECOMMENDATION

Consider Removing referrer assignment from `_updateReferrerOnTransfer`; only allow referrer assignment through explicit actions (`setUserReferrer` or `depositWithReferrer`).

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/vault-contract/commit/f59bf9613437c0ff0ffd899137ee07a28e3a1b4c>

MEDIUM **M-03**

RESOLVED

De-whitelisted Referrers Can Still Claim Fees

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225 https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/mihir/add-avon-adapter/src/VaultV2.sol#L1277 https://github.com/GuardianOrg/vault-contract-team1-176375225872/blob/mihir/add-avon-adapter/src/VaultV2.sol#L1233872/blob/mihir/add-avon-adapter/src/VaultV2.sol#L386
REVIEW	Remediation Review 1

DESCRIPTION

The VaultV2 comment states that "Only whitelisted referrers can receive fee distributions", but the implementation does not enforce this guarantee.

Whitelisting is controlled by `setIsWhitelistedReferrer`, which only flips a boolean and removes the address from the `_activeReferrers` set. It does not clear the referrer's stake (`referrerTotalShares`), does not remove their stake from the global stake counter (`totalReferrerShares`), and does not prevent claiming.

Reward claiming is performed via `claimReferrerFees`, which accrues rewards for sender and transfers shares held by the vault to an arbitrary to address. However, there is no check that `msg.sender` is currently whitelisted. Therefore, a referrer that was whitelisted in the past, accrued rewards, and was later removed from the whitelist can still call `claimReferrerFees` and receive fee shares.

Also, the `_accrueReferrer` uses `referrerTotalShares[referrer]` as stake and does not verify `isWhitelistedReferrer[referrer]`. If a referrer is de-whitelisted after having stake, that stake remains in the system and continues to earn index-based rewards unless the owner separately and correctly calls `clearReferrerData`.

RECOMMENDATION

Require `isWhitelistedReferrer[msg.sender]` in `claimReferrerFees` and, upon de-whitelisting, automatically settle the referrer's accrual and remove their stake from `totalReferrerShares` so they can no longer accrue or claim fee distributions.

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/vault-contract/commit/d14621ff974f245b86d3d3a3e9b031b0fe89ebc0>

LOW **L-01**

RESOLVED

_updateOrders Can Execute During Pause

CATEGORY	Unexpected Behavior
LOCATION	AvonPool.sol
REVIEW	Remediation Review 1

— DESCRIPTION

The `_updateOrders` function is invoked by several setter functions that are protected by a timelock. If the pool is paused at that time and a timelocked setter that calls `_updateOrders` is executed, `_updateOrders` will still run even though the pool is paused. As a result, orders are pushed back into the orderbook while the pool is in a paused state.

— RECOMMENDATION

Ensure that `_updateOrders` does not execute while the pool is paused, or add explicit pause checks to all setter functions that invoke it. This prevents paused pools from republishing orders, especially if the timelock execution role is ever set to be permissionless.

— RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/d52b70f32945575e17cc6b0fa154acf926897b33>

LOW **L-02**

ACKNOWLEDGED

Share Inflation Can Cause Temporary DOS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/0b3254493a44ac54e0f85c85aedaff79e9b6d685/src/pool/extensions/BorrowRepay.sol#L25
REVIEW	Remediation Review 1

— DESCRIPTION

The cleanup logic that previously zeroed totalBorrowShares in _repay was removed to prevent an accounting desynchronization between the global totalBorrowShares and the sum of all user borrowShares. However, this reintroduces the initial borrow-share inflation edge case, which can be abused to temporarily block borrows.

— RECOMMENDATION

Consider enforcing a minimum borrow amount, or a minimum totalBorrowAssets whenever there are open borrows, to prevent manipulation of borrow share math using small values.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

RESOLVED

Misnamed Slippage Parameter In `_borrow`

CATEGORY —

LOCATION <https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/0b3254493a44ac54e0f85c85aedaff79e9b6d685/src/pool/extensions/BorrowRepay.sol#L25>

REVIEW Remediation Review 1

DESCRIPTION

The `_borrow` function applies the slippage guard correctly by ensuring that the computed borrow shares do not exceed the user-provided bound.

However, the parameter is currently named `minExpected`, which is misleading. The check enforces an upper bound on shares, meaning it is actually acting as a maximum acceptable amount of borrow shares rather than a minimum.

RECOMMENDATION

Rename the parameter to reflect its current role as a maximum borrow share amount

RESOLUTION

Status: Resolved.

<https://github.com/avon-xyz/avon-periphery/commit/70343af5ccd210dcd1aef41e4189b56f3ac1bb9b>

INFORMATIONAL I-02

ACKNOWLEDGED

Paused Pool Quotes Can Desync State

CATEGORY	Informational
LOCATION	Orderbook.sol
REVIEW	Remediation Review 1

DESCRIPTION

The orderbook keeps two related states - the lender quote liquidity stored in `lenderTree` and a per-pool list of rates in `poolOrders[pool]` that is primarily maintained when pools call `batchInsertOrder`. These can become inconsistent in pause/refresh edge cases.

When a pool pauses, it attempts to clear its quotes by calling the orderbook with empty arrays inside a `try/catch` block.

```
try IOrderbook(s.orderBook).batchInsertOrder(rates, liquidity) {}  
catch {  
  emit PoolEvents.CancelOrdersFailed(address(this));  
}
```

However, `batchInsertOrder` is gated by `whenNotPaused`. If the orderbook is also paused at that time, the cancellation reverts and the failure is swallowed, leaving the pool's old quotes still present in `lenderTree` and its rates still listed in `poolOrders`.

Once the orderbook is later unpaused (while the pool can remain paused), borrowing matches are allowed again. The matching flow first consumes/removes entries from `lenderTree` via `_matchOrder`, and only afterwards drops paused pools during aggregation (in `_aggregatePoolData`).

This ordering allows a paused pool's quotes to be removed from `lenderTree` without any interaction with the pool, while `poolOrders[pool]` is not pruned to reflect the removals. The inconsistency persists until the pool successfully re-quotes (or an admin removes the pool), and during this window off-chain consumers of `getPoolOrders` may observe stale and unserviceable rates.

RECOMMENDATION

Be aware of this edge case inconsistency and consider making quote cancellation/refresh resilient to orderbook pauses (e.g. allow empty `batchInsertOrder` calls to cancel existing orders even while paused).

I-02 Paused Pool Quotes Can Desync StateCONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Shares Sent Without Receive Gate Checks

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/mihir/add-avon-adapter/src/VaultV2.sol#L1300 https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/mihir/add-avon-adapter/src/VaultV2.sol#L1277
REVIEW	Remediation Review 1

DESCRIPTION

The vault uses gates (`receiveSharesGate` , `sendSharesGate`) to control which accounts can receive or send shares via `canReceiveShares` / `canSendShares` , and these checks are enforced in `enter` , `transfer` , and `transferFrom` . However, `claimReferrerFees` and `sweepOrphanedShares` transfer shares without calling `canReceiveShares` .

As a result, the gating policy is not consistently enforced across all share transfer paths, which can undermine assumptions and lead to shares being held by addresses that should have been blocked.

RECOMMENDATION

Verify if receiver address can receive shares in both `claimReferrerFees` and `sweepOrphanedShares` functions. If the current behavior is intentional, explicitly document that these two functions bypass `receiveShareGate/canReceiveShares`, and note that if a blacklist is introduced in the future, these paths should be updated to add the gate check.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Re-whitelist Restores Stake With Stale Index

CATEGORY Logical Error
LOCATION VaultV2.sol
REVIEW Remediation Review 2

DESCRIPTION

The contract intends to prevent unwhitelisted referrers from later receiving rewards for the period they were not whitelisted by updating their `referrerIndexOf` without accruing rewards. However, when the user is re-whitelisted, `setIsWhitelistedReferrer` adds the preserved stake back to `totalReferrerShares` but does not reset `referrerIndexOf[referrer]` to the current `referrerIndex` and does not call `_accrueReferrer(referrer)` while the referrer is still non-whitelisted.

If `referrerIndex` increases while the referrer is de-whitelisted and their checkpoint remains stale, the referrer can later be re-whitelisted and call `claimReferrerFees`, causing `_accrueReferrer` to credit them for index growth that occurred while they were not eligible, draining referrer escrow and reducing/delaying other referrers' claims.

RECOMMENDATION

In the re-whitelisting path, update the global index and set `referrerIndexOf[referrer]` to the current `referrerIndex` before re-adding their stake, so they start accruing only from the re-whitelisting moment.

RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Withdraw DoS After Referrer De-Whitelist

CATEGORY Logical Error
LOCATION VaultV2.sol
REVIEW Remediation Review 2

DESCRIPTION

When the owner de-whitelists a referrer, `setIsWhitelistedReferrer` subtracts the referrer's entire stake from `totalReferrerShares` but leaves `referrerTotalShares[referrer]`. Later, both `exit` (used for withdraw and redeem) and `_updateReferrerOnTransfer` (used by transfer and transferFrom) can still compute a non-zero stake reduction from the reserved `referrerTotalShares` and decrement `totalReferrerShares` without checking whether the referrer is currently whitelisted. For users tied to that de-whitelisted referrer, this decrement can underflow (since `totalReferrerShares` was already reduced), reverting and preventing withdrawals/redeems and transfers until the referrer is re-whitelisted or the owner clears data.

RECOMMENDATION

Only decrement `totalReferrerShares` when the referrer is currently whitelisted.

RESOLUTION

Status: Resolved.

MEDIUM **M-03**

RESOLVED

totalReferrerShares Drift On Transfers

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/658b776cc1ba190b63f36a4b33d0bcbee26ecab/src/VaultV2.sol#L1153
REVIEW	Remediation Review 2

— DESCRIPTION

totalReferrerShares is intended to track the amount of shares currently counted toward referrer fee distribution. When a referrer is de-whitelisted, their stake is removed from totalReferrerShares, but users can still hold shares pointing to that removed referrer. In `_updateReferrerOnTransfer`, when shares move from a user tied to a removed referrer to a user tied to a whitelisted referrer, those shares should transition from "not counted" to "counted" and increase totalReferrerShares accordingly. However, the function only increases totalReferrerShares via the `missingStake` adjustment, which is typically zero in the de-whitelist case (since `referrerTotalShares[fromReferrer]` still exists and the subtraction does not cap). As a result, totalReferrerShares can remain too low after these transitions, drifting from the true sum of whitelisted referrer stakes. This drift can skew referrer fee allocation and referrer index accounting by using an incorrect global denominator.

— RECOMMENDATION

In `_updateReferrerOnTransfer`, update totalReferrerShares based on whether the transferred shares were previously counted and whether they should be counted after transfer.

— RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

clearReferrerData Skews totalReferrerShares

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/658b776cc1ba190b63f36a4b33d0bcbee26ecab/src/VaultV2.sol#L416
REVIEW	Remediation Review 2

— DESCRIPTION

When a referrer is de-whitelisted, `setIsWhitelistedReferrer(referrer, false)` already removes the referrer's entire stake by subtracting `referrerTotalShares[referrer]` from `totalReferrerShares`. Later, if the owner calls `clearReferrerData(referrer)`, the function reads `stake = referrerTotalShares[referrer]` and conditionally subtracts that same stake from `totalReferrerShares` again whenever `totalReferrerShares >= stake`.

As a result, `clearReferrerData` can double-subtract an already-excluded stake, causing `totalReferrerShares` to drift downward and no longer match the sum of active referrer stake. This can lead to a underflow/DoS in referrer-related transfer and withdrawal logic.

— RECOMMENDATION

Do not decrement `totalReferrerShares` in `clearReferrerData` for referrers whose stake has already been removed on de-whitelisting.

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

Legacy userAttributedShares Wipes New Stake

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/vault-contract-team1-1763752225872/blob/09f9cedd39a20d5032273957e17879fdf6e75a87/src/VaultV2.sol#L387
REVIEW	Remediation Review 3

— DESCRIPTION

When a referrer is removed from the whitelist, the vault deletes `referrerTotalShares[referrer]`, but it does not clear users' `userAttributedShares` tied to that referrer. If the referrer is later re-whitelisted, only new deposits are added back into `referrerTotalShares[referrer]`, while the user's `userAttributedShares` still includes legacy attributed shares from before the removal.

This creates a mismatch after re-whitelisting: the referrer does not earn fees (after being re-whitelisted) from legacy `userAttributedShares` accumulated before/during de-whitelisting because those shares are not included in `referrerTotalShares[referrer]`. However, in `exit()`, the withdrawal reduction amount is still computed from `userAttributedShares`, meaning those legacy attributed shares can still drive reductions that wipe out the referrer's newly re-accrued stake from fresh deposits.

Example:

- User A deposits and has `userAttributedShares = 200` linked to Referrer A.
- Referrer A is de-whitelisted → `referrerTotalShares[A]` is cleared to `0`, but user A still has `userAttributedShares = 200`.
- Referrer A is re-whitelisted and user B deposits, adding `50` new attributed shares → `referrerTotalShares[A] = 50`.
- If user A now withdraws, `exit()` will reduce `referrerTotalShares[A]` by up to `50` (capped), wiping out all of Referrer A's new stake even though the withdrawal was primarily consuming legacy attributed shares that do not earn fees.

This can cause referrers to lose future earnings capacity on legitimate new deposits due to withdrawals of older, non-counted stake.

— RECOMMENDATION

Consider avoiding re-whitelisting the same referrer address after removal. If a referrer must be reinstated, require a new referrer address to prevent legacy `userAttributedShares` from wiping post-reinstatement stake.

M-01 Legacy userAttributedShares Wipes New Stake

CONTINUED

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Liquidation Can Panic With Zero bonusFactor

CATEGORY	Logical Error
LOCATION	https://github.com/avon-xyz/avon-periphery/blob/main/src/pool/extensions/Liquidation.sol#L43 https://github.com/avon-xyz/avon-periphery/blob/main/src/pool/extensions/Liquidation.sol#L83
REVIEW	Remediation Review 3

DESCRIPTION

The liquidation pricing logic can hit an unhandled division by zero edge case where the computed collateral value rounds down to zero while the borrower still has non zero debt, which can occur when

```
position.collateral * collateralPrice < ORACLE_PRICE_SCALE (1e36).
```

In liquidation, the code computes a collateral value in loan-token units using integer division, then derives a `coverageRatio` and clamps the liquidation `bonusFactor` to that ratio to avoid worsening the borrower's health. When `coverageRatio` becomes 0, `bonusFactor` becomes 0 as well, and subsequent calculations divide by `bonusFactor`, triggering the panic revert instead of a controlled protocol error.

RECOMMENDATION

Be aware of this rare dust/low-price scenario and add a check to revert when `bonusFactor` is zero.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Residual Limit Orders Brick on Partial Fill

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/e6fa0740ce360d719198b3ca16bff2153d78d5bf/src/Orderbook.sol#L364
REVIEW	Remediation Review 4

— DESCRIPTION

When a keeper partially fills a borrower limit order, the protocol attempts to preserve the unfilled remainder, but reinserts the residual borrower-tree entry under the wrong account. In `matchLimitBorrowOrder()`, the keeper executes the fill and then calls `_cancelBorrowerOrder()` with only the matched amount. If the order is only partially filled, `_cancelBorrowerOrder()` removes the old tree entry and reinserts the remainder through `borrowerTree._insertOrder(false, rate, ltv, orderAmount - amount)`. However, `_insertOrder()` records `entry.account = msg.sender`, and in this path `msg.sender` is the keeper, not the borrower.

This leaves the residual order in a split-brain state. The stored order metadata remains under `borrowersOrders[borrower]` with reduced amount, `minAmountExpected`, and `collateralAmount`, but the live borrower-tree entry now belongs to the keeper. All later management paths still assume both pieces belong to the borrower. The borrower still has the stored order, but `_cancelBorrowerOrder()` scans the tree for `entry.account == borrower`, which now fails because the residual entry was reassigned to the keeper. As a result, the borrower cannot cancel the remainder, later keeper execution of the remainder fails through the same broken ownership path, and the borrower cannot replace the order at the same (rate, ltv) bucket because that update path also first tries to cancel the old order. In practice, once a keeper partially fills a borrower limit order, the remaining order becomes bricked.

Beyond bricking the residual order, each such partial fill also leaves behind an orphaned borrower-tree entry that normal flows cannot clean up. Since `_cancelBorrowerOrder()` performs a linear scan over all entries in the borrower bucket, these dead entries increase future scan costs for later cancellations, same-bucket replacements, and keeper fills. Over time, repeated partial fills at the same (rate, ltv) can therefore accumulate broken entries, increase gas costs, and worsen the borrower-bucket DOS surface.

— RECOMMENDATION

If the intended behavior is to preserve residual orders after partial keeper fills, the post-fill path must reinsert the remainder under the borrower address rather than under `msg.sender`. More generally, borrower-tree ownership should always be sourced from the logical

H-01 Residual Limit Orders Brick on Partial Fill

CONTINUED

order owner, not from the immediate caller context. If that behavior is not intended, the protocol should instead fully remove partially filled limit orders and avoid leaving any residual order state behind.

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Partial Fills Leave Residual Orders Unbacked

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/e6fa0740ce360d719198b3ca16bff2153d78d5bf/src/Orderbook.sol#L327
REVIEW	Remediation Review 4

— DESCRIPTION

When a keeper partially fills a borrower limit order, the protocol preserves a residual order in storage, but refunds collateral using the stale pre-update collateralAmount rather than the residual order's updated backing. In `matchLimitBorrowOrder()`, the function snapshots `collateralAmount = order.collateralAmount` before calling `_cancelBorrowerOrder()`. During a partial fill, `_cancelBorrowerOrder()` reduces the stored order in place and leaves a residual order with `collateralAmount` proportional to the unfilled amount. However, after returning, `matchLimitBorrowOrder()` still computes `excessCollateral = collateralAmount - totalCollateral` using the original pre-update collateral value, refunds that excess back to the borrower, and then forwards `totalCollateral` to the pool.

This exhausts the full original escrow during the partial fill. The matched portion of collateral is pulled into the pool, and the remainder is refunded back to the borrower. Despite that, the residual order that remains in `borrowersOrders[borrower]` still reports a nonzero `collateralAmount` as if collateral were still backing the remaining order. The protocol therefore reaches a state where stored borrower-order accounting diverges from actual collateral balances held by the orderbook.

In practice, after a partial keeper fill, the orderbook can preserve a residual borrower order whose recorded collateral backing no longer matches actual escrowed funds. Any later logic that treats `order.collateralAmount` as live reserved collateral will be operating on stale accounting rather than real backing.

— RECOMMENDATION

If the intended behavior is to reinsert residual orders after partial keeper fills, collateral refunds must be calculated from the updated residual order state rather than from the stale pre-fill collateral snapshot. The residual order should only remain in storage if the corresponding residual collateral also remains escrowed in the orderbook. If that behavior is not intended, the protocol should instead fully remove partially filled limit orders and avoid leaving any residual collateral state behind.

M-01 Partial Fills Leave Residual Orders Unbacked

CONTINUED

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

ACKNOWLEDGED

Partial Fills Enforce Full-Order Min Output

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/e6fa0740ce360d719198b3ca16bff2153d78d5bf/src/Orderbook.sol#L327
REVIEW	Remediation Review 4

— DESCRIPTION

When a keeper partially fills a borrower limit order, the protocol validates the matched slice against the full pre-fill `minAmountExpected` instead of against a proportional minimum for the portion actually executed. In `matchLimitBorrowOrder()`, the function snapshots `minAmountExpected = order.minAmountExpected` before matching. If the order is only partially filled, the protocol still executes the borrow for only `matchedOrder.totalMatched`, but later checks if `(netAmount < minAmountExpected)` revert `ErrorsLib.InsufficientAmountReceived()`. This compares the proceeds of a partial execution against the minimum output floor for the entire original order.

As a result, legitimate partial fills can revert even when the matched slice executed correctly. For example, if a borrower posts an order for 2000 units with `minAmountExpected = 1800`, and only 1000 units are currently fillable, the partial execution can return at most roughly 1000 units before fees. The protocol nevertheless compares that partial result against 1800 and reverts. In practice, this means partial fills only succeed when the borrower set `minAmountExpected` low enough to tolerate much smaller executions than the original order size.

This is also inconsistent with the protocol's own partial-order accounting model. `_cancelBorrowerOrder()` already prorates `minAmountExpected` during partial reductions by subtracting only the matched portion's proportional minimum from the stored residual order. The matching path, however, still enforces the original full-order minimum against the partial fill itself. The protocol therefore treats `minAmountExpected` as proportional in one partial-order path and absolute in another, causing otherwise valid partial executions to fail.

— RECOMMENDATION

If partial limit-order fills are intended to be supported, `matchLimitBorrowOrder()` should validate `netAmount` against a proportional minimum corresponding only to `matchedOrder.totalMatched`, using the same scaling logic already applied in `_cancelBorrowerOrder()`.

M-02 Partial Fills Enforce Full-Order Min Output

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

No Admin Cancel for Stuck Borrow Orders

CATEGORY Warning
LOCATION Orderbook.sol
REVIEW Remediation Review 4

— DESCRIPTION

The orderbook provides no privileged mechanism to cancel or clear borrower limit orders once normal user flows become impractical or unusable. Borrower-side cleanup is only available through `cancelBorrowOrder()`, keeper execution in `matchLimitBorrowOrder()`, or same-bucket replacement through `insertLimitBorrowOrder()`. If a borrower bucket becomes excessively bloated, stale entries accumulate, or a specific order becomes operationally stuck, there is no owner or emergency-admin function that can force-cancel the order and return the associated collateral.

This reduces the protocol's ability to respond to griefing or stale-order conditions. Affected orders remain dependent on the same broken or impractical path, with no administrative fallback to unwind them and return collateral.

— RECOMMENDATION

Consider allowing a trusted admin or emergency role to call borrower-order cancellation on a user's behalf and refund the related collateral

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

RedStone maxAge allows stale data

CATEGORY Configuration**LOCATION** <https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/main/src/oracle/RedstoneOracle.sol#L50>**REVIEW** Remediation Review 4

— DESCRIPTION

The RedStone oracle adapters set `maxAge` to 600000000 and compare it against `block.timestamp`, what gives a freshness window of roughly 19 years rather than a normal oracle heartbeat window.

As a result, `getCollateralToLoanPrice` can accept stale RedStone prices for borrow previews, collateral safety checks and liquidation logic.

— RECOMMENDATION

Replace the default `maxAge` value with a realistic freshness window.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Loan/USD freshness is not validated

CATEGORY Configuration
LOCATION Oracle
REVIEW Remediation Review 4

DESCRIPTION

`getLoanToUsdPrice` function is part of the shared oracle interface and is implemented by every oracle adapter. However, it returns the loan/USD feed answer without performing any freshness check, and `_getTicks` uses that value to convert pool liquidity into USD before choosing the quote granularity.

As a result, stale loan/USD prices can silently drive `_getTicks()` into the wrong bucket and cause pools to publish an incorrect number and distribution of quotes.

RECOMMENDATION

Make every oracle adapter's `getLoanToUsdPrice` validate `updatedAt` against `maxAge` and revert on stale data.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Underwater Liquidations Lack Incentive

CATEGORY Warning
LOCATION Liquidation.sol
REVIEW Remediation Review 4

— DESCRIPTION

The liquidation logic can make deeply underwater liquidations loss-making for liquidators. The computed liquidation bonus is clamped to the position's collateral coverage ratio via $\text{bonusFactor} = \text{Math.min}(\text{bonusFactor}, \text{coverageRatio})$. When $\text{collateralValue} < \text{borrowedAssets}$, this ratio is below $1e18$, so the effective bonus falls below par.

In this state, a liquidator can be required to repay more loan-token value than the seized collateral is worth. This clears the borrower and avoids pool bad-debt realization, but shifts the shortfall to the liquidator. Rational users are unlikely to execute these liquidations, leaving deeply underwater positions dependent on backstop intervention.

— RECOMMENDATION

Consider explicitly monitoring deeply underwater positions and liquidating them through a protocol-controlled backstop or keeper process, since permissionless users are unlikely to perform liquidations that require them to absorb the shortfall. Alternatively, adjust the liquidation flow so that insufficient collateral is realized as protocol bad debt rather than being pushed onto the liquidator.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Zero repay allows users to discard dust interest

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/47105e6e0df9f7dd3fc83609b62cadbc7d8999a2/src/pool/AvonPool.sol#L799 https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/47105e6e0df9f7dd3fc83609b62cadbc7d8999a2/src/pool/extensions/AccrueInterest.sol#L49-L53 https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/47105e6e0df9f7dd3fc83609b62cadbc7d8999a2/src/pool/AvonPool.sol#L289-L299
REVIEW	Remediation Review 4

DESCRIPTION

The pool accrual logic advances `lastUpdate` even when computed interest rounds down to zero. The public `accrueInterest` enforces a 12-hour cooldown to prevent accrual spam, but repay accepts `(0, 0)` inputs and calls the internal accrual path directly, bypassing that guard. An unprivileged caller can therefore trigger accruals every block, discarding each interval's fractional interest rather than letting it accumulate.

```
uint256 accruedInterest = totalBorrowAssets.mulDiv(expFactor - PoolConstants.WAD,
PoolConstants.WAD);
if (accruedInterest == 0) {
    s.lastUpdate = currentTime;
    return (0, 0);
}
```

This is reachable through `repay(0, 0, onBehalf)`. The repay entrypoint calls `accrueInterest` first, then routes zero inputs into the exact-shares repay path, which completes as a no-op repayment. As a result, an unprivileged caller can repeatedly trigger accrual on short intervals and keep resetting `lastUpdate` whenever the pool accrued interest for that interval is still below 1 smallest unit.

The effect is limited to small pools, low-decimal assets, low rates and short intervals, where fractional interest can be repeatedly discarded and lender yield understated.

RECOMMENDATION

Reject `repay(0, 0, onBehalf)` so users cannot trigger zero-effect accrual updates.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Stale quote can block borrow matching

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/f335b74ae12c569e94be61c1794181763264ef7/src/pool/utils/PoolGetter.sol#L64 https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/5f25f7071b2009828f6bc4d6e2334eb8c418e349/src/Orderbook.sol#L280-L302 https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/5f25f7071b2009828f6bc4d6e2334eb8c418e349/src/Orderbook.sol#L627-L632 https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/5f25f7071b2009828f6bc4d6e2334eb8c418e349/src/Orderbook.sol#L609 https://github.com/GuardianOrg/avon-periphery-team1-1763891033217/blob/f335b74aee12c569e94be61c1794181763264ef7/src/pool/extensions/CollateralManagement.sol#L19
REVIEW	Remediation Review 5

DESCRIPTION

During pool matching, the orderbook selects pools from the lender tree and then retrieve collateral value from each selected pool using `previewBorrow` and finally executes `depositCollateral` and process borrow for each pool.

Pool quotes in the lender tree reflect the pool's state at the time they were written. `_updateOrders` uses the stored `s.totalBorrowAssets` to compute capacity under the borrow cap, while `previewBorrow` uses the projected post-accrual value. As interest accrues and the projection crosses the borrow cap, `previewBorrow` returns 0 for any new borrow while the stale quote remains in the tree.

```
if (s.borrowCap > 0 && previewPool.totalBorrowAssets + assets > s.borrowCap) return 0;
```

When `matchMarketBorrowOrder` runs, it calls `previewBorrow` on each matched pool to compute required collateral. For a cap-breached pool this returns 0, which is forwarded directly into `depositCollateral`, causing an unconditional revert.

```
function _depositCollateral(PoolStorage.PoolState storage s, uint256 assets, address onBehalf)
internal {
    if (assets == 0) revert PoolErrors.ZeroAssets();
```

There is no skip or fallback logic, so the transaction reverts even when other whitelisted pools have available liquidity. Because the transaction reverts atomically, the stale quote is not consumed, and subsequent borrowers can repeatedly hit the same failing path even if other pools are healthy.

RECOMMENDATION

Treat `previewBorrow == 0` as non-executable quote and exclude it from execution before calling `depositCollateral`. Remove or refresh that stale entry in the lender tree so it cannot repeatedly block later market borrows.

PROOF OF CONCEPT <https://github.com/GuardianOrg/avon-core-team1-1763891009755/blob/d7573b7867942f00c1b0b350701f76977914b4f2/test/StaleQuoteCanBlockBorrowMatchingPoC.sol>

M-01 Stale quote can block borrow matching

CONTINUED

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.